# Práctica acumulativa de PHP para un jukebox

## *Un proyecto individual para una asociación*

**RA1. Selecciona las arquitecturas y tecnologías de programación web en entorno servidor, analizando sus capacidades y características propias.**

Formas parte del equipo que desarrolla una aplicación musical para una asociación. El cliente ya dispone de un reproductor realizado con HTML, CSS y JavaScript. Ahora necesita una web donde sus participantes puedan registrarse, iniciar sesión y gestionar el catálogo que escucha el jukebox.

Tu encargo es construir la parte de servidor en PHP e integrar el reproductor que has trabajado en **0612 Desenvolupament Web en Entorn Client**. Cada estudiante desarrolla y prueba su propia aplicación. Al finalizar, el equipo presenta las distintas implementaciones y explica las decisiones tomadas.

## *Qué debe poder hacer el cliente*

**1.** Crear una cuenta de prueba con un alias, un correo ficticio y una contraseña. Entrar y salir de su sesión.

**2.** Acceder a una home privada con el reproductor y enlaces al catálogo musical.

**3.** Consultar, añadir, editar y eliminar fichas de canciones. Estas cuatro operaciones forman el **CRUD**: Create, Read, Update y Delete.

**4.** Conservar usuarios y canciones en archivos JSON aunque se cierre el navegador o se reinicie el contenedor.

El catálogo será compartido: en este prototipo, cualquier participante autenticado podrá gestionarlo. No habrá roles, pagos, recuperación de contraseñas ni subida de archivos. Usaremos cuentas ficticias y audios locales con permiso de uso. Eliminar una ficha no borrará su archivo de audio.

Empieza en **C:\server\asociacion-jukebox-php**. No crees toda la aplicación ahora: cada bloque añade una necesidad al trabajo anterior. El primer entorno usará 8090; la segunda etapa, 8091.

**Continuidad con RA2. Escribe sentencias ejecutables por un servidor web reconociendo y aplicando procedimientos de integración del código en lenguajes de marcas.** Las páginas, formularios y respuestas del proyecto ejercitan esa integración. Los apartados nuevos incluyen una explicación antes de pedir su aplicación.

# Cómo avanzar y conservar el trabajo

Las primeras entregas son pequeñas comprobaciones del futuro catálogo. Después reutilizarás sus datos, condiciones y funciones en las pantallas definitivas. Los ejercicios no cambian de cliente ni de aplicación.

## El recorrido del proyecto

**Bloques 1 a 3.** Arrancas PHP, creas una portada y describes una canción. Calculas su duración y decides qué mensaje mostrar.

**Bloques 4 a 6.** Construyes un catálogo con arrays, separas funciones y recibes datos mediante GET y POST. Aún no guardas cambios.

**Bloques 7 y 8.** Conservas la primera etapa, amplías Docker y lees el catálogo desde un JSON privado. Las bases de datos se prueban por separado; el CRUD seguirá usando JSON.

**Bloques 9 y 10.** Incorporas registro, login y sesión. Después añades creación, edición y eliminación de canciones sobre el mismo catálogo.

**Bloques 11 y 12.** Conectas el jukebox a los datos del servidor, realizas las pruebas finales y presentas el proyecto.

## Antes de dar por terminado un ejercicio

Identifica **qué cambia** y **por qué**. Escribe **qué vas a probar** y **qué resultado esperas**. Programa una parte pequeña, pruébala y explica cualquier diferencia entre tu predicción y lo observado. Las pistas son fragmentos para empezar; debes completar las decisiones indicadas.

Conserva los archivos numerados: 01-bienvenida.php, 02-portada.php, etc. Los números identifican pasos del desarrollo. Más adelante crearás register.php, login.php y home.php porque esas serán las direcciones de uso de la aplicación, no ejercicios independientes.

## Un informe individual desde el primer bloque

Prepara un informe PDF con índice y una sección por bloque. Describe qué pidió el cliente, qué archivos cambiaste, una prueba válida y una prueba de error o límite. Indica la URL o el comando, el resultado esperado, el observado y una explicación natural.

Añade capturas legibles con un pie que explique qué demuestran. No muestres contraseñas, cookies de sesión, tokens ni datos personales. No inventes errores: si no hubo una incidencia, explica una comprobación real.

Guarda un resumen breve en docs/01-entorno.md, docs/02-portada.md y los siguientes. Puede reutilizar la explicación del informe. Conserva una versión comprobada al cerrar cada bloque; si utilizas Git, crea un commit y revisa que no incluyes datos privados.

# Bloque 1 Levantar el primer servicio

**Encargo.** La asociación necesita comprobar que su futura web puede ejecutar PHP. Trabajas con carpetas, Compose, Apache y puertos; todavía no hay usuarios ni canciones guardadas.

## Ejercicio 1 Preparar el proyecto

Abre la carpeta del proyecto en VS Code. Crea compose.yaml, src/01-bienvenida.php y docs/. Comprueba que los archivos no terminan en .txt. En el PHP escribe un saludo con echo que identifique el jukebox de la asociación.

```
// ## Visual Studio Code ##
<?php
// Primer punto de comprobación del servidor.
echo 'Bienvenido al jukebox de la asociación';
```

## Ejercicio 2 Completar y arrancar Compose

Completa el montaje de ./src en /var/www/html. La primera ruta pertenece a Windows; la segunda es la carpeta desde la que Apache sirve tus páginas. Mantén la misma sangría en las claves del servicio.

```
// ## Visual Studio Code ##
name: asociacion-jukebox-inicial
services:
  web:
    image: php:8.5-apache-bookworm
    ports:
      - "127.0.0.1:8090:80"
    # Añade volumes y el montaje de src.
```

```
TERMINAL · PowerShell
cd C:\server\asociacion-jukebox-php # Entra en el proyecto.
docker compose config --quiet # Valida el YAML.
docker compose up -d # Arranca el servicio web.
docker compose ps # Comprueba estado y puerto.
curl.exe -i http://localhost:8090/01-bienvenida.php
# La última orden muestra estado, cabeceras y saludo.
```

**Qué cambia y por qué.** Ahora un servidor ejecuta tu PHP. El navegador entra por 8090 y Docker dirige la petición al puerto 80 del contenedor. No abras el PHP con doble clic: utiliza la URL.

**Comprueba.** Espera un 200 y el saludo correcto. Cambia una palabra, guarda y recarga: no necesitas reconstruir la imagen.

**En el informe.** Captura el saludo y la respuesta HTTP. Dibuja con diagrams.net el navegador, el ordenador con src/ y el contenedor con Apache y PHP; señala montaje y puertos.

**Logro del bloque.** Puedes relacionar el archivo que editas con la respuesta que recibe el navegador.

# Bloque 1 Distinguir errores de acceso

## Ejercicio 3 Comparar dos fallos

Pide http://localhost:8090/no-existe.php con el servicio activo. Después detén el servicio y vuelve a pedir el saludo que sí existe. No cambies el archivo para esta prueba.

```
TERMINAL · PowerShell
docker compose stop # Detiene este proyecto.
curl.exe -i http://localhost:8090/01-bienvenida.php
# Ahora no habrá respuesta HTTP del servicio detenido.
docker compose start # Recupera los contenedores.
docker compose logs --tail=20 web # Lee sus registros.
```

**Comprueba.** Un 404 demuestra que un servidor respondió, aunque no encontró la ruta. Con el servicio detenido falla la conexión. Termina verificando otra vez un 200 y el saludo. Añade al informe una tabla con acción, resultado e interpretación.

## Ejercicio 4 Revisar la sintaxis del PHP

Escribe una segunda instrucción echo. Quita temporalmente el punto y coma de la primera y revisa la sintaxis. Hazlo entre dos instrucciones PHP, no justo antes de una etiqueta de cierre.

```
TERMINAL · PowerShell
docker compose exec web php -l /var/www/html/01-bienvenida.php
# Revisa la sintaxis del archivo dentro del contenedor.
```

Lee el archivo y la línea señalados; la causa puede estar en la línea anterior. Restaura el símbolo y repite la orden. Debe aparecer No syntax errors detected. Después pide la página: una sintaxis correcta no garantiza que el contenido cumpla el encargo.

## Conservar esta primera entrega

Guarda compose.yaml y el saludo como referencia. En los siguientes bloques crearás nuevos PHP en src/. No necesitas cambiar el Compose para añadir una página ni instalar PHP en Windows.

Usa localhost cuando trabajes en tu ordenador. Con la publicación en 127.0.0.1, otro equipo del aula no puede entrar por su propia dirección localhost ni por la IP de tu equipo. El acceso de red se razonará al comparar arquitecturas.

**En el informe.** Explica la diferencia entre error de conexión, 404 y error de sintaxis. Basta una captura legible del diagnóstico y una prueba final correcta. No copies el registro entero.

# Bloque 2 Describir la primera canción

**Encargo.** La asociación quiere presentar su jukebox y destacar una canción. Usarás PHP dentro de HTML, variables, tipos, constantes y concatenación. Consulta esas secciones de IntroduccionPHP.pdf.

## Ejercicio 1 Crear una portada

Crea src/02-portada.php con una estructura HTML completa, lang="es", meta charset="utf-8", un h1 y un párrafo generado con echo. Enlaza el saludo del bloque 1. Explica que el catálogo está en preparación; no muestres botones de login que todavía no funcionan.

**Comprueba.** En Ver código fuente aparecen HTML y el saludo, pero no la instrucción echo. El resultado de PHP llega al navegador; el programa permanece en el servidor.

## Ejercicio 2 Reutilizar los datos de una canción

Crea src/03-cancion.php. Mantendremos estas claves al construir el catálogo. duration expresa segundos y src es una URL relativa a la web, no una ruta C:\ de Windows.

```
// ## Visual Studio Code ##
<?php
$title = 'Sesión de prueba'; // Texto ficticio.
$artist = 'Grupo del aula';
$duration = 185; // Segundos enteros.
$volume = 0.5; // Decimal para explorar tipos.
$available = true; // Booleano de prueba.
const APP_NAME = 'Jukebox de la asociación';
// Muestra title en un encabezado y en una frase.
```

Usa var_dump temporalmente para comprobar cada tipo y comparar 185 con '185'. Después retíralo. Cambia solo $title y comprueba sus dos apariciones. La constante identifica la aplicación; no se reasigna durante la ejecución.

## Ejercicio 3 Preparar la salida HTML

Prueba el título ficticio Demo. Aplica htmlspecialchars al mostrarlo con las opciones ENT_QUOTES | ENT_SUBSTITUTE y UTF-8. El dato guardado sigue siendo el original; lo que preparas es su representación en HTML.

**Comprueba.** Debes ver las etiquetas como texto, sin convertir Demo en negrita.

**En el informe.** Incluye una tabla de tipos, el cambio del título en dos lugares y el título escapado.

**Logro del bloque.** La futura ficha se genera desde datos, no desde textos duplicados.

# Bloque 3 Calcular y decidir sobre una canción

**Encargo.** El cliente quiere mostrar una duración comprensible y distinguir canciones cortas y largas. Crea src/04-duracion.php. Trabaja primero con enteros no negativos; las entradas externas llegarán después.

## Ejercicio 1 Separar minutos y segundos

Una duración de 185 segundos debe mostrarse como 3 minutos y 5 segundos. intdiv divide dos enteros y devuelve el cociente entero; % devuelve el resto. Aplica ambas operaciones a la misma duración, sin escribir el resultado a mano.

```
// ## Visual Studio Code ##
<?php
$duration = 185;
$minutes = intdiv($duration, 60); // Minutos completos.
$seconds = $duration % 60; // Segundos restantes.
// Construye y muestra la frase con ambos valores.
```

Predice qué obtendrás para 0, 59, 60 y 185. Comprueba cada caso. No es necesario formatear todavía 03:05: primero debes entender los dos números.

## Ejercicio 2 Aplicar una regla del cliente

En src/05-estado-cancion.php, muestra un único estado: con duración 0, «Sin duración»; de 1 a 240 segundos, «Hasta cuatro minutos»; por encima de 240, «Más de cuatro minutos». Programa if, elseif y else con llaves {}.

```
// ## Visual Studio Code ##
if ($duration === 0) {
    // Prepara el mensaje de este caso.
}
// Completa las otras dos ramas, en ese orden.
```

Dibuja el flujo de decisiones y prueba 0, 1, 239, 240 y 241. Compara además 0 === '0' fuera del cálculo normal: explica por qué no son idénticos. No conviertas la duración a texto para conseguir un mensaje.

**Qué cambia y por qué.** La ficha ya interpreta sus datos. Estas reglas servirán para mostrar las canciones del catálogo y para revisar los datos que lleguen de un formulario.

**Comprueba.** 240 pertenece al segundo caso y 241 al tercero. Cada petición muestra solo un estado.

**En el informe.** Incluye el diagrama, la tabla de duraciones y una captura del límite 240. Distingue =, que asigna, de ===, que compara valor y tipo.

**Logro del bloque.** Puedes justificar el mensaje de una canción mediante una condición y sus pruebas.

# Bloque 4 Construir el catálogo musical

**Encargo.** La asociación quiere pasar de una canción a varias sin repetir el HTML de cada ficha. Trabajas con for, while, arrays y foreach.

## Ejercicio 1 Preparar una lista sencilla

En src/06-listas.php, genera con for una lista de posiciones del 1 al 3 y otra igual con while. Usa llaves. Traza contador, condición y salida, incluida la última condición falsa. Cambia el límite a 4 y comprueba ambas listas.

Después crea un array con tres títulos y recórrelo con foreach. Su primer índice es 0. No confundas el número mostrado al usuario con la posición interna del array.

## Ejercicio 2 Añadir los datos de cada ficha

Crea src/07-catalogo.php. Transforma la lista de títulos en una colección de arrays asociativos. Este será el formato compartido por PHP, JSON y el jukebox; conserva los nombres de las claves.

```
// ## Visual Studio Code ##
$songs = [
    [
        'id' => 1,
        'title' => 'Sesión de prueba',
        'artist' => 'Grupo del aula',
        'category' => 'electronica',
        'duration' => 185,
        'src' => 'audio/demo-01.mp3',
    ],
    // Añade dos canciones con identificadores distintos.
];
```

Usa las categorías electronica, rock y bandas-sonoras. Los archivos de audio se incorporarán al integrar el reproductor; por ahora src es un dato de la ficha. El id identifica una canción y no debe cambiar aunque cambie su posición.

Recorre la colección, muestra título, artista y duración, y escapa los textos para HTML. Añade una cuarta canción cambiando únicamente los datos. No escribas un echo adicional para ella.

**Comprueba.** Con cuatro registros aparecen cuatro fichas. Cada id es distinto. Si la colección está vacía, muestra «Todavía no hay canciones».

**En el informe.** Relaciona una ficha, un elemento del array y una vuelta del bucle. Añade la traza del contador y el caso de catálogo vacío.

**Logro del bloque.** Tu catálogo aumenta al añadir datos, sin duplicar su presentación.

# Bloque 5 Separar funciones y datos

**Encargo.** El equipo necesita reutilizar el cálculo de duración en varias pantallas. Ahora completarás un encargo con tres pasos sobre el mismo catálogo.

## Paso 1 Probar una función pequeña

En src/08-funciones.php define duracionLegible(int $total): string. Debe devolver una frase con minutos y segundos usando el cálculo del bloque 3. No debe imprimir HTML. Llama a la función con 60 y 185 y muestra ambos resultados.

```
// ## Visual Studio Code ##
function duracionLegible(int $total): string
{
    // Calcula minutos y segundos.
    // Devuelve el texto con return, sin echo.
}
```

El parámetro recibe un valor para cada llamada. El tipo int indica el dato esperado y : string indica que devolverás texto. En esta prueba usa duraciones no negativas.

## Paso 2 Mover la regla a un archivo común

Mueve la definición a src/lib/canciones.php. Cárgala desde 08-funciones.php y desde 07-catalogo.php mediante require_once. Esta instrucción carga el archivo una sola vez durante la petición; evita redefinir una función al reutilizar componentes.

```
// ## Visual Studio Code ##
require_once __DIR__ . '/lib/canciones.php';
// __DIR__ es la carpeta del archivo que contiene esta línea.
// El punto une esa ruta con el nombre del archivo común.
```

Mueve también el array del catálogo a src/lib/catalogo-inicial.php. Haz que ese archivo devuelva la colección con return y recíbela con $songs = require __DIR__ . '/lib/catalogo-inicial.php';. La biblioteca de funciones y los datos tienen responsabilidades distintas.

## Paso 3 Comprobar una dependencia

Cambia temporalmente el nombre de una ruta de require, pide la página y consulta docker compose logs --tail=30 web. Después restaura la ruta. No borres el archivo real ni cambies Docker para corregir una ruta PHP.

**Comprueba.** La función devuelve 1 minuto y 0 segundos para 60, y 3 minutos y 5 segundos para 185. El catálogo sigue mostrando todas sus fichas.

**En el informe.** Señala definición, llamada y retorno. Explica por qué php -l puede aceptar la página aunque el archivo requerido no exista.

**Logro del bloque.** El siguiente bloque podrá usar el catálogo y sus reglas sin copiarlos otra vez.

# Bloque 6 Recibir una petición del navegador

**Encargo.** El cliente quiere filtrar el catálogo por categoría. Introducimos GET: PHP recibe en $_GET los parámetros que aparecen después del signo ? en la URL.

## Paso 1 Filtrar sin cambiar los datos

Crea src/09-filtro.php. Carga el array común. Si no llega category o vale todas, muestra todas las canciones. Para una categoría admitida, muestra solo sus canciones. La colección guardada no cambia por filtrar la pantalla.

```
// ## Visual Studio Code ##
$category = $_GET['category'] ?? 'todas';
$allowed = ['todas', 'electronica', 'rock', 'bandas-sonoras'];
// ?? usa el valor de la derecha si el dato no está definido.
// Primero valida el tipo con is_string.
// Después comprueba in_array($category, $allowed, true).
```

Si la categoría es desconocida o llega como array, responde con estado 400 y «Categoría no válida», sin listar canciones. Llama a http_response_code(400) antes del HTML. Una categoría válida sin canciones devuelve 200 con un mensaje de lista vacía.

## Paso 2 Probar casos diferentes

Abre /09-filtro.php sin parámetros, con ?category=rock, con ?category=desconocida y con ?category[]=rock. Los corchetes hacen que PHP reciba un array: comprobar solo si el valor está vacío no basta.

```
TERMINAL · PowerShell
curl.exe -g -i "http://localhost:8090/09-filtro.php?category[]=rock"
# -g permite enviar los corchetes sin que curl los interprete.
```

**Qué cambia y por qué.** Ahora la entrada no la escribe tu programa. Comprueba el tipo antes de llamar a funciones de texto, y los valores admitidos antes de filtrar. El parámetro category no se utiliza para construir rutas de archivos.

**Comprueba.** La petición inválida devuelve 400; una categoría vacía válida devuelve 200. No aparecen Warning ni Fatal error.

**En el informe.** Incluye los cuatro casos con estado esperado y observado. Señala dónde se lee, dónde se valida y dónde se usa la categoría.

**Logro del bloque.** Puedes responder de forma controlada a datos que no elegiste tú.

# Bloque 6 Preparar un formulario de canción

**Encargo.** La asociación quiere revisar una ficha antes de guardarla. Crea src/10-formulario.php. Este paso es una **vista previa**: todavía no modifica el catálogo ni escribe archivos.

## Paso 3 Distinguir abrir y enviar

Crea un formulario con method="post" y campos title, artist, category, duration y src. El atributo name determina la clave recibida en $_POST. POST envía los datos en el cuerpo de la petición; no los hace seguros por sí solo.

```
// ## Visual Studio Code ##
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $title = $_POST['title'] ?? '';
    // Comprueba que es texto antes de aplicar trim.
    // Prepara errores o la ficha de vista previa.
}
```

Al abrir la página por GET, muestra el formulario sin errores. Al enviar, exige título y artista no vacíos, una categoría admitida y duración entera entre 1 y 1800 segundos. Para src usa provisionalmente un select con audio/demo-01.mp3 y audio/demo-02.mp3; no aceptes una ruta libre.

## Paso 4 Validar y mostrar sin guardar

Los controles HTML ayudan, pero PHP debe repetir la validación. Para duration, comprueba primero que llega texto y utiliza FILTER_VALIDATE_INT con filter_var; un resultado false significa que no es un entero válido. Después comprueba el intervalo. No conviertas directamente 'abc' a int: perderías la causa del error.

Si hay errores, muestra mensajes junto a los campos y conserva los textos válidos escapados. Si no los hay, muestra la vista previa usando duracionLegible. Indica expresamente «Vista previa, canción no guardada».

Prueba una ficha válida, título vacío, espacios, duración abc, 0 y 1801. Prueba también Demo como título y un campo enviado como array. El título debe verse literalmente; la entrada array se rechaza sin avisos PHP.

**Comprueba.** El catálogo sigue igual después de recargarlo. Un POST inválido puede devolver 400 y el formulario con sus mensajes; un POST válido devuelve la vista previa.

**En el informe.** Captura una ficha válida y un error. En Red o Network, localiza POST y sus campos, sin capturar datos sensibles.

**Logro del bloque.** Has preparado la validación que reutilizarás al crear y editar canciones.

# Bloque 7 Preparar la segunda etapa de Docker

**Encargo.** Antes de guardar información, el equipo quiere separar el código público de los datos privados y comparar dos motores para una futura evolución. Consulta DockerCompose-Dockerfile.pdf, especialmente las secciones de build, Apache, environment y healthcheck.

## Paso 1 Conservar y continuar

Desde ahora conserva sin cambios el Compose de la raíz y su src/. Crea docker/ y copia allí src/ completo, incluida lib/. Trabajarás en esa copia, en el puerto 8091. No copies datos reales de otras aplicaciones.

```
// ## Visual Studio Code ##
asociacion-jukebox-php/
  compose.yaml          # Etapa inicial, 8090.
  src/                  # Pasos 01 a 10 y lib/.
  docs/
  docker/
    compose.yaml        # Nueva etapa, 8091.
    Dockerfile
    .env.example
    apache/jukebox.conf
    src/                # Copia y continuación del código.
    data/               # JSON privados, fuera de src/.
```

Crea docker/data/ aunque esté vacía. El nuevo servicio web debe montar ./src en /var/www/html y ./data en /var/www/data. Apache servirá únicamente /var/www/html. Escribir una URL /data/usuarios.json no debe dar acceso a ese archivo privado.

## Paso 2 Adaptar el Dockerfile y el sitio

Parte de php:8.5-apache-bookworm y añade pdo_pgsql y pdo_mysql como en la teoría. Son controladores de PHP, no los motores de base de datos. Copia docker/apache/jukebox.conf a sites-available y actívalo con a2ensite jukebox, desactivando 000-default.

En jukebox.conf utiliza ServerName jukebox.test, ServerAlias localhost y DocumentRoot /var/www/html. Conserva el bloque Directory y los logs de la teoría. a2ensite activa un sitio; a2enconf activa una configuración general, como jukebox-servername.conf. Ninguna de esas órdenes crea un servicio de Compose.

Excluye .env y docker/data/ del repositorio y de la entrega de datos. En .dockerignore excluye también esas rutas para no enviarlas al contexto de construcción. Los JSON de ejemplo estarán aparte y contendrán solo datos ficticios.

**Comprueba.** COPY debe apuntar a un archivo existente respecto al contexto de construcción. El nombre del virtual host no registra un DNS en Windows. Primero probarás con localhost:8091.

# Bloque 7 Comprobar los servicios y los montajes

## Paso 3 Completar el nuevo Compose

Usa name: asociacion-jukebox-multiservicio. Define web con build.context: .. y build.dockerfile: docker/Dockerfile, publicación 127.0.0.1:8091:80 y los dos montajes de la página anterior. No añadas container_name.

Añade postgres:17-bookworm y mariadb:11.4. Usa las bases jukebox_pg y jukebox_maria, el usuario jukebox y un volumen por motor. Pasa las claves desde .env y conserva una .env.example ficticia. Completa los healthcheck y depends_on con service_healthy explicados en la teoría; no publiques 5432 ni 3306 en Windows.

```
TERMINAL · PowerShell
cd C:\server\asociacion-jukebox-php\docker # Nueva etapa.
docker compose config --quiet # Valida sin mostrar claves.
docker compose up -d --build # Construye y arranca.
docker compose ps # Revisa servicios y salud.
docker compose exec web apache2ctl -t # Sintaxis Apache.
docker compose exec web apache2ctl -S # Sitio activo.
docker compose exec web php -m # Busca pdo_pgsql y pdo_mysql.
```

## Paso 4 Separar conexión y almacenamiento

En docker/src/11-conexiones.php, prueba primero una conexión PDO a PostgreSQL y después otra a MariaDB. Lee la configuración con getenv. Consulta SELECT current_database() y SELECT DATABASE(), respectivamente. Devuelve JSON con estado y nombre de base, nunca claves; usa 503 si falla alguna conexión y registra el diagnóstico sin secretos.

PDO abre la conexión; query ejecuta la consulta y fetchColumn recoge su primer valor. Rodea esta prueba con try y catch (PDOException $error). Este inicio corresponde a PostgreSQL:

```
// ## Visual Studio Code ##
$dsn = 'pgsql:host=postgres;port=5432;dbname=jukebox_pg';
$db = new PDO($dsn, getenv('PGUSER'), getenv('PGPASSWORD'));
$name = $db->query('SELECT current_database()')->fetchColumn();
```

**Las canciones y los usuarios no se guardarán en estas bases.** El CRUD de esta entrega usa archivos JSON. Esta comprobación permite identificar servicios y valorar otra arquitectura; no requiere tablas, SQL de inserción ni migración de datos.

**Comprueba.** Espera tres servicios, bases healthy, Apache Syntax OK y una respuesta PDO 200 con dos estados ok. Healthy no valida por sí solo las credenciales de PHP.

**En el informe.** Muestra ambas etapas, el esquema de servicios y la prueba PDO. Distingue los bind mounts del código y JSON de los volúmenes de los motores. No ejecutes down -v.

**Logro del bloque.** La segunda etapa funciona sin alterar la primera y mantiene los datos fuera de la carpeta pública.

# Bloque 8 Leer el catálogo desde JSON

**Encargo.** El catálogo ya no debe estar escrito dentro de un PHP. Crea docker/data/canciones.json con las fichas del bloque 4. En esta etapa lo editarás a mano solo para preparar datos de prueba; las pantallas de escritura llegarán después.

## Paso 1 Reconocer el formato

JSON representa datos como texto. Usa comillas dobles en claves y cadenas, no escribas comentarios ni dejes coma tras el último elemento. La raíz será una lista. id y duration serán números; las demás claves contendrán texto.

```
// ## Visual Studio Code ##
[
  {
    "id": 1,
    "title": "Sesión de prueba",
    "artist": "Grupo del aula",
    "category": "electronica",
    "duration": 185,
    "src": "audio/demo-01.mp3"
  }
]
```

## Paso 2 Cargar y mostrar la colección

Crea docker/src/lib/almacen-json.php y docker/src/12-catalogo-json.php. La página carga la colección y reutiliza el HTML y las funciones del catálogo. En el contenedor, la ruta fija será /var/www/data/canciones.json, nunca una ruta enviada por el navegador.

```
// ## Visual Studio Code ##
$text = file_get_contents('/var/www/data/canciones.json');
// Comprueba $text === false antes de decodificar.
$songs = json_decode($text, true, 512, JSON_THROW_ON_ERROR);
// true convierte los objetos JSON en arrays asociativos.
// Comprueba la lista y los campos antes de mostrarlos.
```

file_get_contents lee texto. json_decode lo convierte en datos PHP. JSON_THROW_ON_ERROR permite tratar JSON mal formado con catch (JsonException $error). Ante un fallo de lectura o formato, muestra un mensaje controlado y no escribas nada. No sustituyas un archivo dañado por una lista vacía.

**Comprueba.** Añadir una ficha al JSON hace aparecer otra ficha en la página. [] es un catálogo vacío válido. Una coma incorrecta en una copia de prueba debe producir un error controlado, no pérdida de datos. Restaura la copia al terminar.

**En el informe.** Explica archivo JSON, array PHP y HTML resultante. Verifica que /data/canciones.json no descarga los datos por HTTP.

**Logro del bloque.** El catálogo tiene una única fuente de datos fuera de la carpeta pública.

# Bloque 9 Preparar sesiones y formularios privados

**Encargo.** Antes de guardar usuarios, necesitas comprender cómo se conserva la identificación entre peticiones. Una sesión guarda datos en el servidor y el navegador envía una cookie con su identificador. No guardes la contraseña en la sesión ni en JavaScript.

## Paso 1 Preparar una sesión común

Crea docker/src/lib/sesion.php y cárgalo con require_once antes de cualquier HTML en register.php, login.php, home.php y las acciones privadas. session_start abre o recupera la sesión. Configura antes session.use_strict_mode=1 y la cookie con httponly=true y samesite=Lax; secure será true cuando exista HTTPS. Este laboratorio local utiliza HTTP y no debe publicarse en Internet.

En esta sesión guardarás más adelante usuario_id y alias tras verificar las credenciales. En una prueba de lectura, una sesión sin usuario_id significa que todavía no hay una persona autenticada. No simules el login asignando un usuario fijo en home.php.

```
// ## Visual Studio Code ##
ini_set('session.use_strict_mode', '1');
session_name('JUKEBOXSESSID'); // Nombre propio para este proyecto.
session_set_cookie_params(['path' => '/', 'httponly' => true,
    'samesite' => 'Lax', 'secure' => false]); // HTTP local.
session_start(); // Antes de generar HTML.
```

## Paso 2 Preparar la protección de los envíos

Los formularios que modifican datos llevarán un token CSRF: un texto aleatorio asociado a la sesión. PHP comparará el token enviado con el guardado para rechazar envíos que no procedan de uno de sus formularios. Esta comprobación no sustituye la validación de los campos ni el login.

```
// ## Visual Studio Code ##
if (!isset($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}
// Envía este valor en un input hidden llamado csrf.
// Escápalo al insertarlo como atributo HTML.
```

En cada POST, comprueba que csrf es texto y compáralo con hash_equals($_SESSION['csrf'], $recibido). Si falta o no coincide, responde 403 y no escribas. Mantén el token en todos los formularios, incluidos registro, login y logout. No lo incluyas en las capturas.

## Qué probar antes de continuar

Muestra un formulario de prueba con su token. Envía normalmente y verifica que se acepta la comprobación. Después elimina el campo oculto desde las herramientas del navegador: el envío debe rechazarse con 403. Aún no se guarda ningún dato.

**En el informe.** Explica qué se conserva en el servidor, qué envía la cookie y para qué se comprueba el token. No copies sus valores. Estas funciones son nuevas: debes poder localizar dónde se inician y dónde se utilizan.

# Bloque 9 Registrar una cuenta de prueba

**Encargo.** La asociación acepta un alias, un correo ficticio y una contraseña de prueba. Crea register.php y docker/data/usuarios.json, inicialmente con []. No reutilices contraseñas personales. El formulario reutiliza lo aprendido con POST y la protección del paso anterior.

## Paso 3 Validar antes de guardar

Exige alias de 2 a 30 caracteres, correo con formato válido y contraseña de 12 a 64 caracteres; repite la contraseña. Comprueba primero que llegan como texto. Aplica trim al alias y al correo, no a la contraseña. Usa FILTER_VALIDATE_EMAIL con filter_var para el correo y mb_strlen para caracteres. Rechaza también contraseñas de más de 72 bytes con strlen: evita el límite de bcrypt al usar PASSWORD_DEFAULT.

Normaliza el correo con strtolower tras trim y recorre usuarios.json para rechazar duplicados. El campo de confirmación no se guarda. Ante errores, conserva alias y correo escapados, pero deja vacíos los dos campos de contraseña.

## Paso 4 Guardar una representación protegida

```php
// ## Visual Studio Code ##
$hash = password_hash($password, PASSWORD_DEFAULT);
// Guarda $hash en password_hash, nunca $password.
$user = [
    'id' => $newId,
    'alias' => $alias,
    'email' => $email,
    'password_hash' => $hash,
];
// Añade el usuario y guarda solo si no hay errores.
```

password_hash genera una representación para verificar contraseñas; no es una contraseña que debas descifrar. El id será un entero generado por el servidor: máximo existente + 1, o 1 si no hay usuarios. La comprobación de duplicados y el alta se harán dentro de la misma operación protegida de archivo que se explica en la página siguiente.

Tras guardar correctamente, redirige a login.php con estado 303 y termina con exit. La redirección hace que la siguiente petición sea GET y evita repetir el POST al recargar. Si la escritura falla, no anuncies «Registro completado».

**Comprueba.** Prueba un registro válido, el mismo correo con mayúsculas, claves diferentes y un campo como array. Solo el válido crea un registro. El JSON contiene password_hash, no la clave original.

**En el informe.** Documenta los casos sin mostrar contraseñas ni hashes completos. Comprueba que /data/usuarios.json no es descargable.

**Logro del bloque.** La cuenta queda guardada y preparada para verificar un login real.

# Bloque 9 Guardar JSON de forma controlada

Esta es una ayuda nueva para el registro y el CRUD. Centraliza la lectura y escritura en lib/almacen-json.php. No repitas un procedimiento distinto en cada formulario y no escribas rutas recibidas por GET o POST.

## Del dato PHP al archivo

json_encode convierte el array a texto JSON. Utiliza JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR para obtener un archivo legible y detectar fallos. La conversión debe terminar correctamente antes de cambiar el archivo existente.

```
// ## Visual Studio Code ##
$json = json_encode(
    $records,
    JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
        | JSON_THROW_ON_ERROR
);
// $json es texto. Todavía no se ha guardado en el disco.
```

## Un procedimiento común de actualización

**1.** Abre la ruta fija existente con fopen en modo r+: permite leer y escribir sin vaciar el archivo al abrirlo. Si falla, detén la operación. Prepara los archivos con [] antes de utilizarlos; no los crees a ciegas tras un fallo.

**2.** Adquiere flock con LOCK_EX y comprueba que lo consigue. Con ese bloqueo, lee con stream_get_contents, decodifica y valida. Revisa duplicados o localiza el id sobre estos datos recién leídos.

**3.** Modifica el array y conviértelo a JSON. Con el mismo bloqueo, vuelve al inicio con rewind, escribe todo el texto con fwrite y comprueba los bytes escritos. Recorta lo sobrante con ftruncate y confirma fflush. No declares éxito si alguna operación falla.

**4.** Libera el bloqueo con LOCK_UN y cierra con fclose, también si hubo un error. Las lecturas del catálogo utilizarán LOCK_SH mientras leen, para no observar una escritura incompleta.

Bloquear solo la escritura final con file_put_contents(..., LOCK_EX) no protege el ciclo completo de leer, modificar y guardar: dos peticiones podrían calcular cambios sobre la misma versión. En este laboratorio se practica el procedimiento completo con ayuda del docente, antes de conectarlo a los formularios.

**Límite del prototipo.** El bloqueo coordina peticiones que usan este mismo procedimiento, pero no garantiza recuperar el archivo ante un corte durante la escritura. Conserva copias de prueba fuera de src/. Una aplicación publicada necesitaría almacenamiento y medidas de recuperación más completos.

**Comprueba.** Un JSON inválido debe abortar sin sobrescribirse. Dos altas sucesivas conservan ambas fichas. Reinicia el servicio y comprueba que siguen ahí. Si hay un error de permisos, revisa el montaje y el usuario de PHP; no apliques permisos 777 como solución general.

# Bloque 9 Entrar y salir de la home

**Encargo.** Implementa login.php, home.php y logout.php. Un registro no da acceso por sí solo: debes verificar una cuenta existente. En home.php muestra primero un saludo con el alias y un enlace al catálogo; el reproductor se integrará al final.

## Paso 5 Verificar las credenciales

En login.php valida el método, el token y los tipos de correo y contraseña. Normaliza el correo igual que en register.php, busca su usuario y comprueba la clave con password_verify. No compares un hash recién creado con el guardado: password_hash puede producir representaciones distintas para una misma clave.

```
// ## Visual Studio Code ##
if (password_verify($password, $user['password_hash'])) {
    session_regenerate_id(true); // Renueva el identificador.
    $_SESSION['usuario_id'] = $user['id'];
    $_SESSION['alias'] = $user['alias'];
    header('Location: home.php', true, 303);
    exit;
}
// Antes de este fragmento debes haber encontrado $user.
```

Para correo desconocido o contraseña incorrecta, muestra el mismo mensaje: «Credenciales no válidas». No reveles cuál de los dos datos falló. No registres la contraseña en logs.

## Paso 6 Comprobar el acceso en el servidor

Crea lib/acceso.php para comprobar usuario_id tras iniciar la sesión. Inclúyelo al principio de home.php y de cada página privada, antes del HTML. Si no hay sesión autenticada, redirige a login.php y termina con exit. Ocultar enlaces no protege una URL escrita directamente.

logout.php aceptará solo POST con token válido. Vacía $_SESSION, elimina la cookie de sesión con los mismos parámetros usados al crearla y ejecuta session_destroy. Redirige a login.php. Un enlace GET no debe cerrar la sesión.

**Comprueba.** En una ventana privada, home.php conduce al login. Con credenciales válidas aparece el alias. Tras salir, volver a pedir home.php exige entrar otra vez. Prueba también URL directa al catálogo y un POST sin sesión.

**En el informe.** Dibuja registro, login, home y logout, indicando cuándo se guarda el usuario y cuándo se abre la sesión.

**Logro del bloque.** El acceso depende de una comprobación PHP en cada petición privada, no del estado de un botón.

# Bloque 10 Consultar y añadir canciones

**Encargo integrado.** La asociación quiere administrar su catálogo desde la web. Reutiliza el JSON del bloque 8, la validación del bloque 6 y las funciones compartidas. Todo el CRUD exige sesión; todos los cambios exigen POST y token válido.

## Paso 1 Consultar el catálogo

Crea canciones.php a partir de 12-catalogo-json.php, añadiendo la comprobación de acceso. Muestra las fichas y enlaces para editar. Incluye un formulario de eliminación por ficha, pero no lo actives hasta completar ese paso. Añade enlaces a home.php y cancion-nueva.php.

Antes del CRUD, protege también 12-catalogo-json.php o deja su versión demostrativa usando datos ficticios independientes. Ninguna página anterior de la segunda etapa debe ofrecer acceso alternativo al catálogo privado. La primera etapa permanece congelada y no lee los JSON actuales.

## Paso 2 Crear una canción

En cancion-nueva.php, reutiliza el formulario de vista previa. Mantén título y artista obligatorios, categorías permitidas, duración de 1 a 1800 y src elegido de una lista de audios admitidos. Limita título y artista a 80 caracteres. La validación del servidor debe ser la misma al crear y al editar.

Si todo es válido, lee la colección bajo bloqueo, calcula el máximo id existente y añade 1; si está vacía, comienza en 1. No uses count($songs) + 1: podría repetir un id después de una eliminación. No aceptes un id propuesto por el formulario de alta.

```
// ## Visual Studio Code ##
$newSong = [
    'id' => $newId, // Lo calcula PHP, no el navegador.
    'title' => $title,
    // Completa artist, category, duration y src.
];
// Añade la ficha a la colección validada.
// Guarda con el procedimiento común y comprueba el éxito.
```

Tras guardar, redirige con 303 a canciones.php y termina con exit. Esta será la operación Create. La lista será Read. No necesitas una API de escritura ni peticiones fetch para estos formularios: PHP puede procesar POST y devolver otra página.

**Comprueba.** Añadir una canción aumenta en uno la colección. Recargar después de la redirección no duplica el alta. Un campo inválido o un token incorrecto no cambia el JSON.

**En el informe.** Incluye la ficha antes y después, su id y una prueba rechazada.

**Logro del bloque.** Las altas modifican la misma fuente que utiliza el catálogo.

# Bloque 10 Editar y eliminar por identificador

## Paso 3 Editar una ficha existente

Crea cancion-editar.php. GET recibe id para cargar el formulario, no para guardar. Comprueba que llega un entero positivo; si es inválido devuelve 400 y si no existe devuelve 404. Recorre la colección hasta encontrar el registro cuyo id coincida, no el elemento situado en esa posición.

Muestra los valores actuales escapados. En el POST, vuelve a validar sesión, token, id y campos. Lee de nuevo la colección bajo bloqueo y localiza otra vez la ficha: no te fíes de que existiera cuando se abrió el formulario. Sustituye sus datos conservando el id y los demás registros.

```
// ## Visual Studio Code ##
foreach ($songs as $position => $song) {
    if ($song['id'] === $id) {
        // $position localiza el elemento del array.
        // $id identifica la canción de forma estable.
        // Actualiza los campos validados, no su id.
    }
}
```

## Paso 4 Confirmar y eliminar

Crea cancion-eliminar.php. Puedes mostrar una confirmación por GET con el título de la ficha, pero el borrado se ejecutará únicamente por POST. Incluye el id y el token en el formulario. Cancelar devuelve al catálogo sin escribir nada.

Con la sesión y los datos comprobados, vuelve a localizar el id bajo bloqueo. Usa unset para quitar ese elemento y array_values para reconstruir índices consecutivos antes de json_encode. Sin reindexar, una lista con huecos puede serializarse como objeto JSON y dejar de servir al reproductor.

La operación elimina solo la ficha. No uses unlink con src ni borres el MP3. Si el id no existe, devuelve 404 sin modificar el catálogo. Ante un método no permitido, responde 405; no borres mediante un enlace directo.

**Comprueba.** Con ids 1, 2 y 3, borra el 2 y edita el 3: debe cambiar la canción correcta. El siguiente alta utiliza 4. El JSON continúa siendo una lista. GET y Cancelar no eliminan.

**En el informe.** Demuestra Update y Delete con una tabla de ids y una captura de confirmación. Explica por qué id y posición no son lo mismo.

**Logro del bloque.** El CRUD está completo sobre una única colección persistente.

# Bloque 11 Conectar el jukebox a la home

**Encargo.** Integra el reproductor que ya has trabajado en la asignatura de cliente. Conserva una copia de sus archivos antes de adaptarlos. No necesitas rehacer el diseño ni el ecualizador: el cambio principal será de dónde recibe las canciones.

## Paso 1 Ubicar los archivos del cliente

Copia CSS, JavaScript y audios autorizados en docker/src/assets/, docker/src/scripts/ y docker/src/audio/, adaptando las rutas de tu proyecto. Inserta el HTML del reproductor dentro de home.php, después de comprobar la sesión. Conserva los ids y clases que utiliza su JavaScript.

El archivo de apoyo equilizer.js busca un canvas con id="equalizer", utiliza Howl y Howler y espera variables como songs e i. Por sí solo no define todo el reproductor. Carga Howler antes del código que lo utiliza y ejecuta el script cuando exista el canvas; puedes conservar el orden de tu jukebox y utilizar defer donde corresponda.

## Paso 2 Ofrecer solo las canciones

Crea docker/src/api/canciones.php. Inicia la sesión, comprueba usuario_id y lee canciones.json mediante la función compartida. Devuelve 200 con Content-Type application/json y una **lista JSON** de canciones, sin envolverla en una clave songs. Así el cliente podrá acceder a songs[i].src.

```
// ## Visual Studio Code ##
header('Content-Type: application/json; charset=utf-8');
// Si falta sesión: responde 401 con un error JSON y exit.
// Lee el catálogo privado con el procedimiento común.
echo json_encode($songs, JSON_UNESCAPED_UNICODE
    | JSON_THROW_ON_ERROR);
```

La API no redirige al login: un fetch que espera JSON no debe recibir una página HTML. No añadas var_dump ni texto antes de header. La respuesta nunca incluye usuarios, hashes, tokens ni rutas internas del contenedor.

Como el endpoint está en api/, sus require necesitan subir una carpeta: __DIR__ . '/../lib/almacen-json.php'. Revisa también la ruta de la biblioteca de sesión. La URL de src se resolverá desde home.php: audio/demo-01.mp3 apunta a un recurso de la web.

**Comprueba.** Con sesión, la API devuelve una lista y el campo src de cada ficha. Sin sesión, devuelve 401 y un error JSON. En ambos casos revisa estado, Content-Type y cuerpo.

# Bloque 11 Probar la integración del reproductor

## Paso 3 Cambiar la fuente de canciones

En la carga de datos del jukebox, sustituye el fetch de ./scripts/songsData.json por ./api/canciones.php. La petición se hará desde la misma web y puerto que home.php. El JSON privado no se descarga directamente: PHP decide qué datos devuelve.

```
// ## Visual Studio Code ##
const response = await fetch('./api/canciones.php');
// Comprueba response.ok antes de continuar.
const songs = await response.json();
// Si no es una lista válida, muestra un error controlado.
// Si está vacía, desactiva reproducción y explica el motivo.
// Solo entonces selecciona una canción y crea su Howl.
```

Adapta las variables a las que ya usa tu reproductor: no declares una segunda songs local si otras funciones dependen de la colección compartida. Define el índice inicial, por ejemplo i = 0, antes de consultar songs[i]. No intentes crear un Howl si la lista está vacía.

Usa title y artist para los textos de la interfaz y src para el audio. Si tu reproductor emplea otros nombres, cambia esa lectura de forma explícita o crea una adaptación documentada. Para texto recibido utiliza textContent, no innerHTML. Una ruta almacenada no demuestra que el audio exista: comprueba la petición del MP3 en Red.

## Paso 4 Completar una prueba de principio a fin

**1.** Entra, crea una ficha con uno de los audios admitidos y vuelve a cargar home.php. Debe aparecer en el reproductor.

**2.** Edita su título y recarga la home. La lista debe mostrar el nuevo valor. El cliente no mantiene otro songsData.json que contradiga al servidor.

**3.** Elimina la ficha y recarga. Ya no debe aparecer. Si no queda ninguna canción, muestra el estado vacío sin errores de índice ni reproducción.

**4.** Pulsa Reproducir. El navegador puede impedir el sonido automático: iniciar audio mediante una acción del usuario forma parte de la prueba. Comprueba también pausa y cambio de canción.

No reinicies el analizador ni añadas otra animación cada vez que cambias de pista si tu reproductor ya tiene esos recursos creados. Revisa la consola y conserva el comportamiento del jukebox que habías probado en cliente.

**En el informe.** Captura la home y la petición a la API, sin cookies. Explica el recorrido PHP lee JSON, responde datos, JavaScript crea la interfaz y Howler reproduce el audio. La pantalla y la API requieren sesión; en este prototipo los archivos estáticos de audio no llevan control de acceso propio.

# Bloque 12 Verificar la entrega completa

**Encargo.** El cliente necesita saber qué funciona y cómo repetir las pruebas. Realiza la comprobación sobre tu proyecto, no sobre capturas del ejemplo. En la raíz prepara un README con requisitos, carpetas, arranque, URL y forma de crear la primera cuenta ficticia.

## Pruebas de aceptación

**1.** Entorno: la etapa inicial responde en 8090 y la ampliada en 8091. Identificas el Compose activo, los montajes, los puertos y las dos conexiones PDO.

**2.** Registro: una cuenta válida se guarda una sola vez; correo duplicado, array y contraseña no confirmada se rechazan. No hay claves en texto plano ni descarga HTTP de usuarios.json.

**3.** Acceso: home, catálogo, acciones y API comprueban sesión. Logout la invalida. Prueba las URLs directamente en una ventana privada, no solo mediante los botones.

**4.** CRUD: alta, lectura, edición y eliminación afectan a la ficha correcta. GET no modifica datos. Falta de token o token incorrecto produce 403 sin escritura; el id inexistente se trata sin alterar otros registros.

**5.** Persistencia: detén y arranca web. El catálogo y las cuentas ficticias continúan en docker/data/. Puede ser necesario iniciar sesión otra vez; persistir usuarios no significa persistir una sesión para siempre.

**6.** Integración: la home recibe el catálogo del endpoint, reproduce un audio permitido y trata la lista vacía. No mantiene una segunda colección independiente en JavaScript.

Guarda una matriz con prueba, preparación, resultado esperado, resultado real y evidencia. Si algo falla, anota el paso pendiente; no lo marques como correcto por haber obtenido 200 en otra página.

## Preparar una copia compartible

Entrega código comentado, README, .env.example, resúmenes Markdown e informe PDF. En data-ejemplo/ incluye canciones.json con datos ficticios compatibles con tus audios y usuarios.json con []. Explica que deben copiarse a docker/data/ solo al preparar una copia nueva, nunca sobre datos que se quieran conservar.

No entregues .env, cookies, tokens, usuarios reales ni copias privadas. Las contraseñas de laboratorio se crean en el equipo de destino. Si usas Git, revisa status, diff y log; que un archivo aparezca en .gitignore no elimina una copia ya seguida por Git.

**Logro.** Otra persona puede arrancar una copia limpia, registrar una cuenta y completar el recorrido sin conocer tu ordenador.

# Bloque 12 Presentar y defender el proyecto

Preparad una presentación técnica en grupo. Cada integrante mostrará una parte verificable de su propia implementación individual. Ensayad con una cuenta ficticia; si utilizáis una captura como respaldo, identificadla como captura, no como ejecución en directo.

## Argumentos de la presentación

**Arquitectura.** Explicad por qué utilizáis Compose, qué prepara el Dockerfile y qué hace Apache. Señalad quién recibe HTTP, quién ejecuta PHP y dónde se guarda cada tipo de dato. Comparad brevemente con Nginx y PHP-FPM, sin necesidad de desplegar otra variante.

**Petición y respuesta.** Seguid una petición concreta desde el navegador hasta un archivo PHP y de vuelta. Identificad puerto, ruta, método, estado, Content-Type y cuerpo. Relacionad una prueba con el log correspondiente.

**Datos y acceso.** Mostrad cómo una canción pasa de JSON a array PHP y vuelve a guardarse. Explicad id frente a posición, validación frente a escape, hash de contraseña frente a sesión, y por qué los datos privados están fuera de DocumentRoot.

**Integración.** Realizad una modificación del catálogo y mostradla en el jukebox tras recargar. Explicad qué parte pertenece a PHP y qué parte a JavaScript. Justificad que las bases se han comparado, pero el CRUD entregado utiliza JSON.

## Una comprobación que pueda repetir otra persona

Entrega a un compañero una URL o comando, la carpeta desde la que debe ejecutarlo y el resultado esperado. Debe poder repetirlo en tu equipo preparado o en su copia, siguiendo el README. Localhost se refiere siempre al ordenador que hace la petición.

## De las presentaciones al examen final de RA1

**Las distintas presentaciones servirán para elaborar las preguntas del examen final de RA1.** Cada grupo propondrá dos preguntas con situación, pregunta, respuesta razonada, evidencia para comprobarla y bloque relacionado. El profesorado las revisará y podrá reformularlas.

Ejemplo: «Una canción sigue apareciendo en el jukebox después de borrarla del catálogo. ¿Qué comprobarías?». La respuesta debe distinguir la colección del servidor, la respuesta del endpoint y los datos que JavaScript conserva en memoria. Una captura aislada no permite decidir cuál falla.

Añade al informe el reparto de intervenciones, una decisión justificada de tu proyecto y una idea técnica aprendida al escuchar otro grupo. Se evaluará comprender y demostrar, no memorizar comandos ni el nombre exacto de un archivo.

# Guía de archivos y consultas de apoyo

## La aplicación al terminar

La raíz conserva compose.yaml, src/ con los pasos 01 a 10 y docs/. En docker/ quedan Compose, Dockerfile, apache/, .env local y data/. La carpeta src/ de esa segunda etapa conserva los pasos y añade las pantallas de uso.

```
// ## Visual Studio Code ##
docker/src/
  11-conexiones.php      # Diagnóstico de los motores.
  12-catalogo-json.php   # Lectura, protegida al añadir login.
  register.php          # Alta de una cuenta.
  login.php             # Verificación de credenciales.
  home.php              # Entrada privada y jukebox.
  logout.php            # Salida mediante POST.
  canciones.php         # Consulta del catálogo.
  cancion-nueva.php      # Alta de ficha.
  cancion-editar.php     # Consulta y cambio por id.
  cancion-eliminar.php   # Confirmación y borrado por POST.
  api/canciones.php      # Lista JSON para el reproductor.
  lib/                  # Funciones, sesión, acceso y datos.
  scripts/              # JavaScript del jukebox.
  assets/               # CSS y otros recursos de interfaz.
  audio/                # Audios admitidos y autorizados.
docker/data/            # canciones.json y usuarios.json.
data-ejemplo/           # Datos ficticios para una copia nueva.
```

Los PHP numerados muestran pasos de aprendizaje; las pantallas finales reutilizan sus reglas. No dupliques la definición de una función en cada pantalla. No añadas otra carpeta pública para compartir los JSON privados.

## Material que acompaña el trabajo

En **IntroduccionPHP.pdf**, consulta sintaxis y variables, páginas 11 a 19; operadores y decisiones, 20 a 23; bucles, 23 a 25; funciones, 26 y 27; arrays, 28 y 29. Para el entorno de esta práctica utiliza Docker, con las rutas y puertos del enunciado.

En **DockerCompose-Dockerfile.pdf**, consulta Dockerfile y Apache, páginas 2 a 7; montajes, variables y servicios, 8 a 13; salud y diagnóstico, 14 a 19. Las partes nuevas de JSON, sesiones y autenticación se explican junto a los ejercicios que las necesitan.

El apoyo equilizer.js se integra con el jukebox realizado en cliente. Material de la asignatura: 0612 Desenvolupament Web en Entorn Client en Moodle.

Referencia técnica: manual oficial de PHP sobre JSON, sesiones, password_hash, password_verify y flock. Consulta una función para entender su responsabilidad, no para copiar una aplicación completa.